

CS 452 Operating systems

Virtual Memory

Dr. Denton Bobeldyk

Virtual Memory

- ❖ Allocation of Frames
- ❖ Thrashing
- ❖ Memory-Mapped Files
- ❖ Allocating Kernel Memory
- ❖ Other Considerations

Virtual Memory

- ❖ Allocation of Frames
- ❖ Thrashing
- ❖ Memory-Mapped Files
- ❖ Allocating Kernel Memory
- ❖ Other Considerations

Allocation of Frames

- ❖ How many frames are allocated to a process?
- ❖ A simple allocation of frames would be:
 - ❖ Initially all pages not used by the OS are on a free-list
 - ❖ Once a page-fault occurs, assign a page from the free-list

Minimum Number of Frames

- ❖ Allocate the minimum number of frames to each process
- ❖ The minimum number of frames is hardware dependent
 - ❖ How many pages could a single instruction straddle/require

Allocation Algorithms

- ❖ Equal Allocation
 - ❖ Con:
 - ❖ Could have a very large and a very small process that are awarded frames equally

Allocation Algorithms

- ❖ Proportional Allocation - Allocate memory based on the **size** of each process

$$S = \sum s_i \quad \text{where } s_i \text{ is the size of virtual memory for process } p_i$$

- ❖ Allocate each process: $a_i = s_i/S \times m$

Where m is the total number of frames available.

Also, sum can't exceed m

Allocation Algorithms

- ❖ Proportional Allocation - Allocate memory based on the **priority** of each process

$$P = \sum p_i \quad \text{where } s_i \text{ is the size of virtual memory for process } p_i$$

- ❖ Allocate each process: $a_i = p_i / S \times m$

Where m is the total number of frames available.

Also, sum can't exceed m

Global vs Local Allocation

- ❖ Local replacement - each process can only select from it's set of allocated frames
- ❖ Global replacement - each process can select from all processes' allocated frames

Global vs Local Allocation Alternative

- ❖ Each process can only select from its set of allocated frames or other processes with lower priority than its own.
- ❖ May cause lower priority processes to perform inconsistently (depending on how many frames were poached by other processes)
- ❖ Global tends to outperform Local allocation.

Non-Uniform Memory Access

- ❖ Systems may have memory that are lower latency for each CPU
- ❖ Can group memory & CPUs by creating **lgroups**.
- ❖ Systems will attempt to allocate memory in 'nearyby' lgroups

Virtual Memory

- ❖ Allocation of Frames
- ❖ Thrashing
- ❖ Memory-Mapped Files
- ❖ Allocating Kernel Memory
- ❖ Other Considerations

Thrashing



Image retrieved from:

<https://www.newsweek.com/tony-hawk-interview-50-skateboarding-legend-talks-dancer-fame-simpsons-915750>

Operating System Essentials Silberschatz / Galvin / Gagne

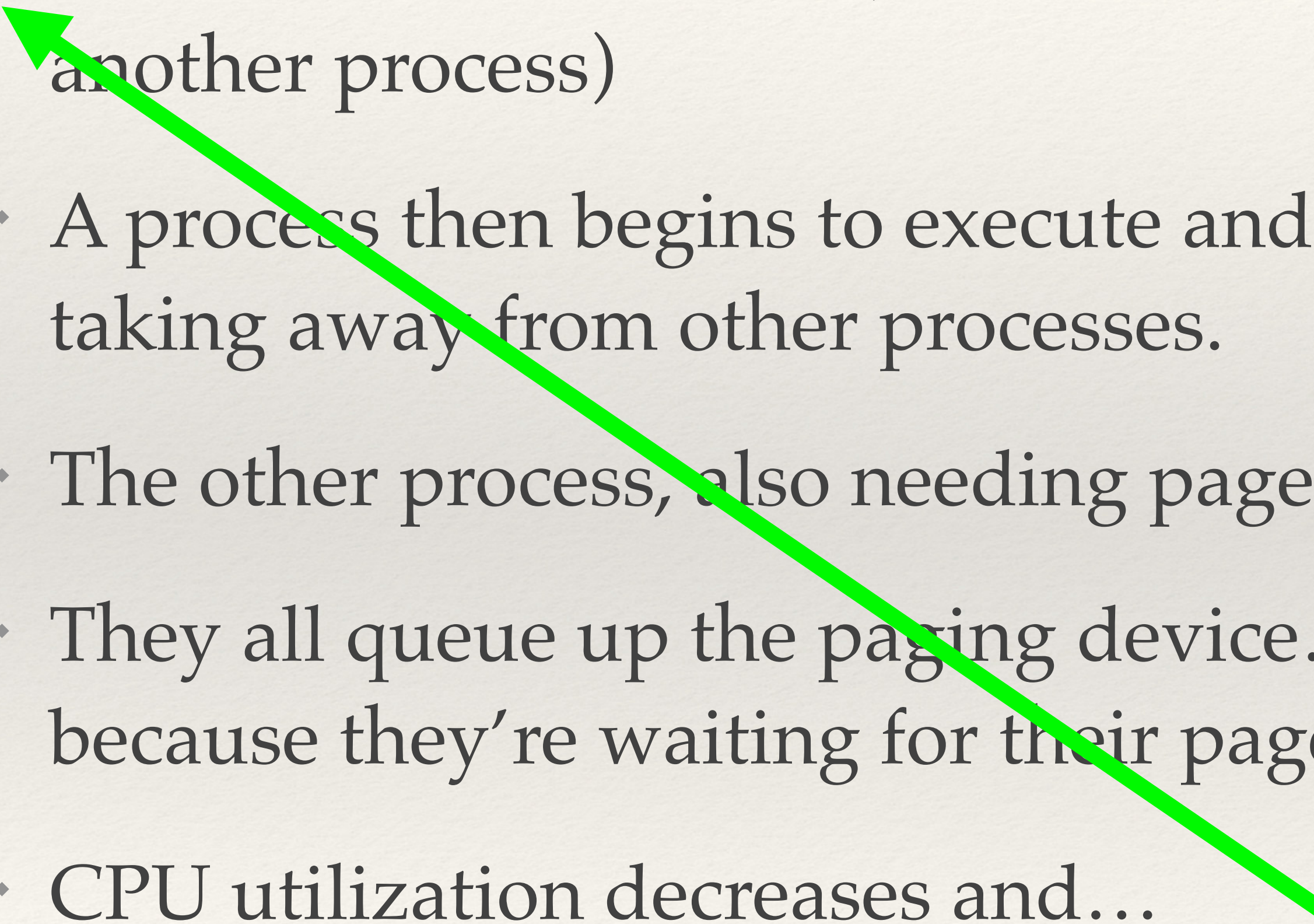
Thrashing

- ❖ What happens when a low priority process falls below the minimum number required by the computer architecture?
- ❖ The process should be suspended
- ❖ Remaining pages swapped out, freeing up all it's allocated frames
- ❖ Why?
 - ❖ The process will continue to page-fault repeatedly...

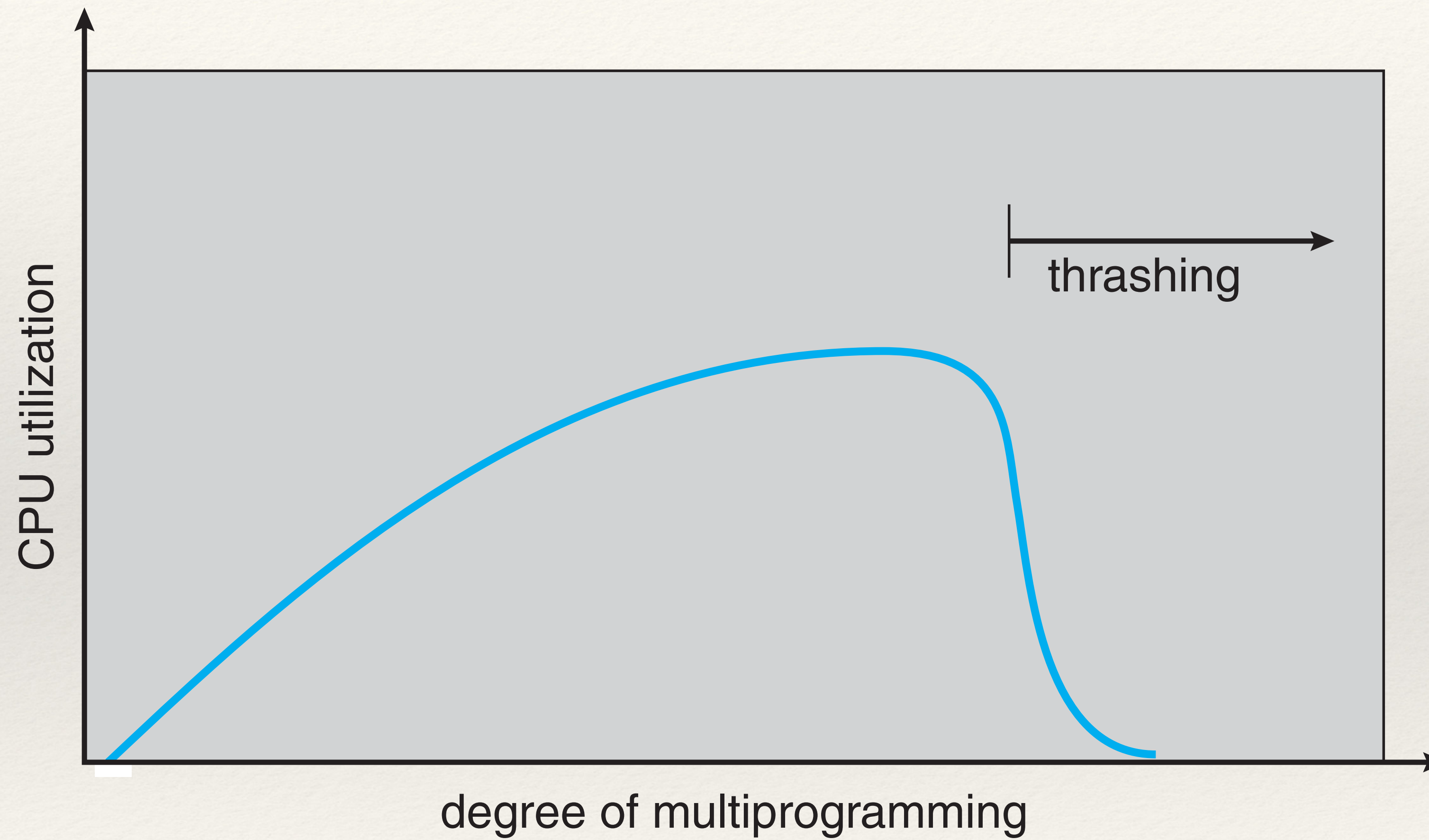
Thrashing

- ❖ If CPU utilization is low, increase the degree of multiprogramming (load another process)
- ❖ A process then begins to execute and needs more frames, page-faulting and taking away from other processes.
- ❖ The other process, also needing pages, page-fault
- ❖ They all queue up the paging device... while the ready queue empties because they're waiting for their page faults to execute...
- ❖ CPU utilization decreases and...

Thrashing

- ❖ If CPU utilization is low, increase the degree of multiprogramming (load another process)
 - ❖ A process then begins to execute and needs more frames, page-faulting and taking away from other processes.
 - ❖ The other process, also needing pages, page-fault
 - ❖ They all queue up the paging device... while the ready queue empties because they're waiting for their page faults to execute...
 - ❖ CPU utilization decreases and...
- 

Thrashing



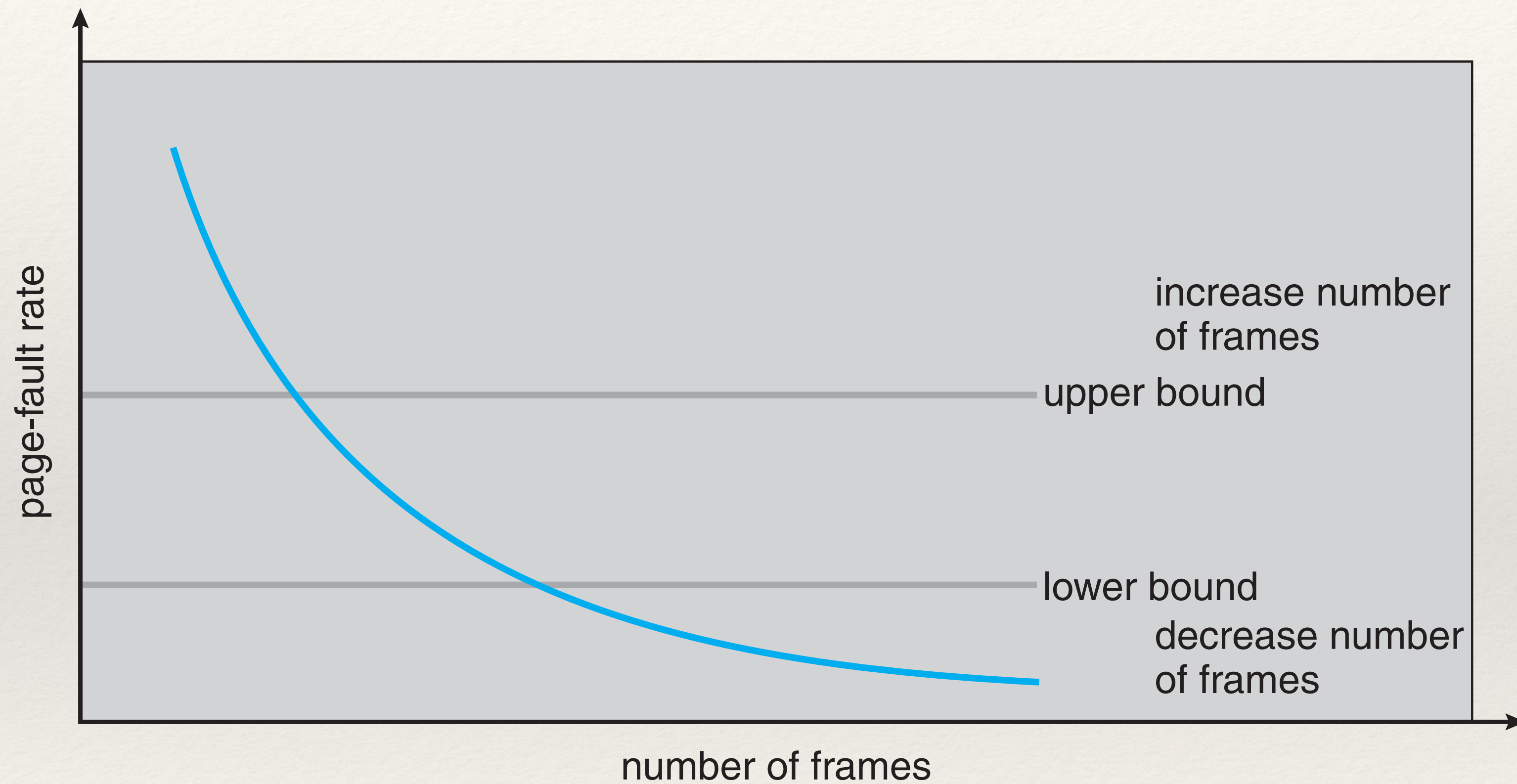
Thrashing

- ❖ Local replacement algorithm can help with thrashing
- ❖ Cannot steal frames from another process
- ❖ But, we already discussed global replacement performs better...

Thrashing - Page Fault Frequency

- ❖ As the page-fault rate increases, we know processes need more frames
- ❖ As the page-fault rate decreases, we know processes have enough frames

Thrashing - Page Fault Frequency



Operating System Essentials / Silberschatz / Galvin / Gagne
May need to swap a process out to stay within the bounds

Thrashing



Image retrieved from:

<https://www.newsweek.com/tony-hawk-interview-50-skateboarding-legend-talks-dancer-fame-simpsons-915750>

Operating System Essentials Silberschatz / Galvin / Gagne

Virtual Memory

- ❖ Allocation of Frames
- ❖ Thrashing
- ❖ Memory-Mapped Files
- ❖ Allocating Kernel Memory
- ❖ Other Considerations

Memory-Mapped Files

- ❖ Mapping a disk block to a page(s) in memory
- ❖ Initial file access results in a page-fault (like normal), however a page-sized portion of the file is read into memory.
 - ❖ Can read in more than one page-sized chunk
- ❖ Subsequent reads / writes are handled as routine memory access increasing performance.

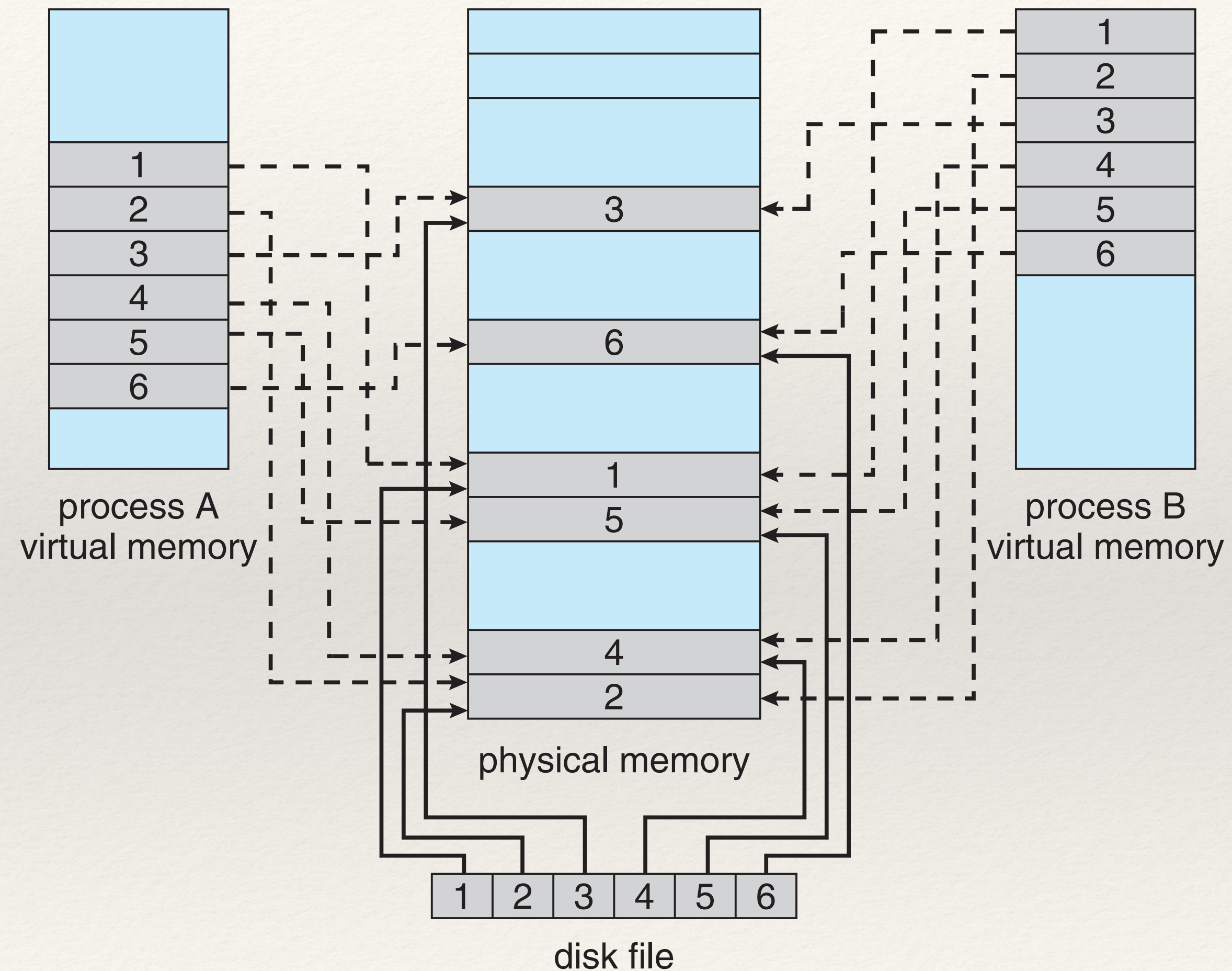
Memory-Mapped Files

- ❖ OS can update the physical file when
 - ❖ it periodically checks whether the page in memory was modified.
 - ❖ The file is closed and all the memory-mapped data is written back to disk

Memory-Mapped Files

- ❖ A file can be specified as memory-mapped using the `mmap()` system call.
- ❖ The operating system can also choose to memory-map a file using standard calls (open, read, write)

Memory-Mapped Files - Memory Sharing



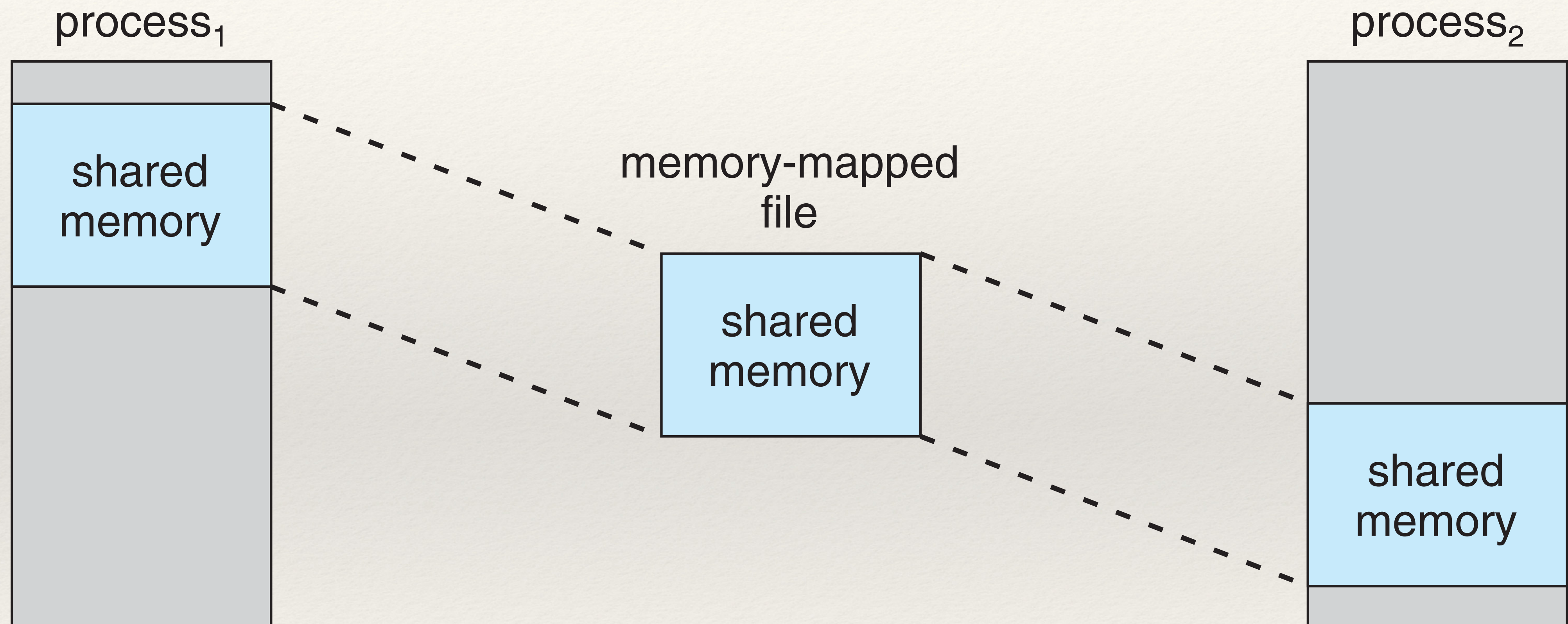
Operating System Essentials Silberschatz / Galvin / Gagne

Multiple processes can memory map the same file

Memory-Mapped Files - Memory Sharing

- ❖ Processes can communicate via shared memory utilizing memory-mapping
- ❖ The memory-mapped file serves as the region of shared memory between the communicating processes

Memory-Mapped Files - Memory Sharing



Shared Memory in the Windows API Overview

1. Create a file mapping for a file to be memory mapped
2. Establish a view of the mapped file in a processes' address space
3. A second process can then open and create a view of the mapped file in its virtual address space

Shared Memory in the Windows API Overview

1. Open the file to be mapped using `CreateFile()` which returns a `HANDLE`
2. Create a memory mapping of this file `HANDLE` using `CreateFileMapping()`

Creates a named shared-memory object

3. Create a view of the mapped file with `MapViewOfFile()`

Producer - Windows API

```
#include <windows.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    HANDLE hFile, hMapFile;
    LPVOID lpMapAddress;

    hFile = CreateFile("temp.txt", /* file name */
        GENERIC_READ | GENERIC_WRITE, /* read/write access */
        0, /* no sharing of the file */
        NULL, /* default security */
        OPEN_ALWAYS, /* open new or existing file */
        FILE_ATTRIBUTE_NORMAL, /* routine file attributes */
        NULL); /* no file template */
```

Producer - Windows API

```
hMapFile = CreateFileMapping(hFile, /* file handle */  
    NULL, /* default security */  
    PAGE_READWRITE, /* read/write access to mapped pages */  
    0, /* map entire file */  
    0,  
    TEXT("SharedObject")); /* named shared memory object */
```

Producer - Windows API

```
lpMapAddress = MapViewOfFile(hMapFile, /* mapped object handle */  
    FILE_MAP_ALL_ACCESS, /* read/write access */  
    0, /* mapped view of entire file */  
    0,  
    0);  
  
/* write to shared memory */  
sprintf(lpMapAddress, "Shared memory message");  
  
UnmapViewOfFile(lpMapAddress);  
CloseHandle(hFile);  
CloseHandle(hMapFile);
```


Consumer – Windows API

```
#include <windows.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    HANDLE hMapFile;
    LPVOID lpMapAddress;

    hMapFile = OpenFileMapping(FILE_MAP_ALL_ACCESS, /* R/W access */
        FALSE, /* no inheritance */
        TEXT("SharedObject")); /* name of mapped file object */

    lpMapAddress = MapViewOfFile(hMapFile, /* mapped object handle */
        FILE_MAP_ALL_ACCESS, /* read/write access */
        0, /* mapped view of entire file */
        0,
        0);

    /* read from shared memory */
    printf("Read message %s", lpMapAddress);

    UnmapViewOfFile(lpMapAddress);
    CloseHandle(hMapFile);
}
```

Virtual Memory

- ❖ Allocation of Frames
- ❖ Thrashing
- ❖ Memory-Mapped Files
- ❖ [Allocating Kernel Memory](#)
- ❖ Other Considerations

Allocating Kernel Memory

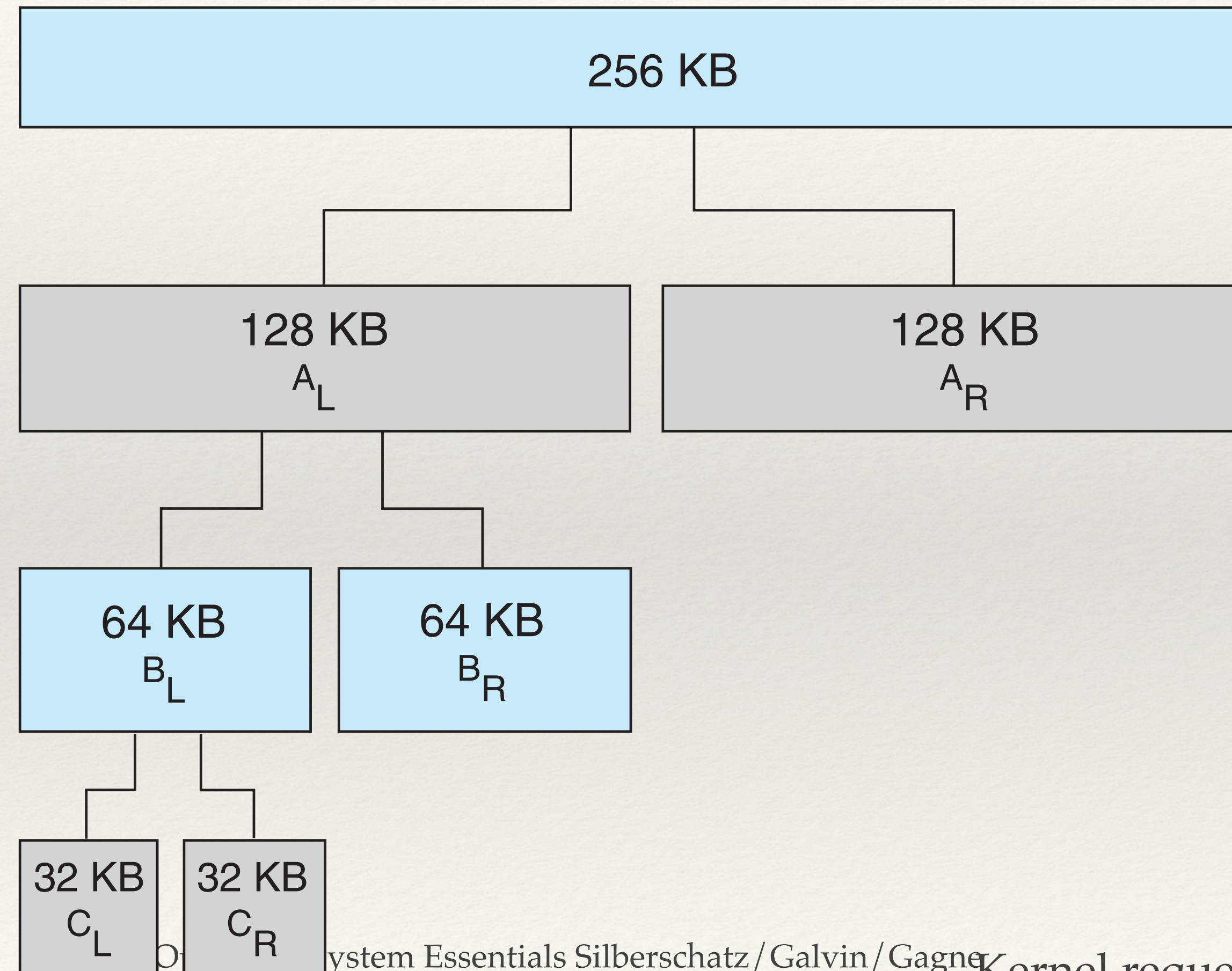
- ❖ Kernel memory is often allocated differently than user requested memory
- ❖ Kernel memory requests can vary in size for different data structures, some of those data structures are less than the page size
- ❖ Certain hardware devices interact directly with physical memory which may require memory residing in contiguous pages

Buddy System

- ❖ Allocates memory from a fixed-size segment consisting of physically contiguous pages.
- ❖ Allocated using a power-of-2 allocator, which satisfies requests as a power of 2 (4KB, 8KB, 16KB, etc)

Buddy System

physically contiguous pages



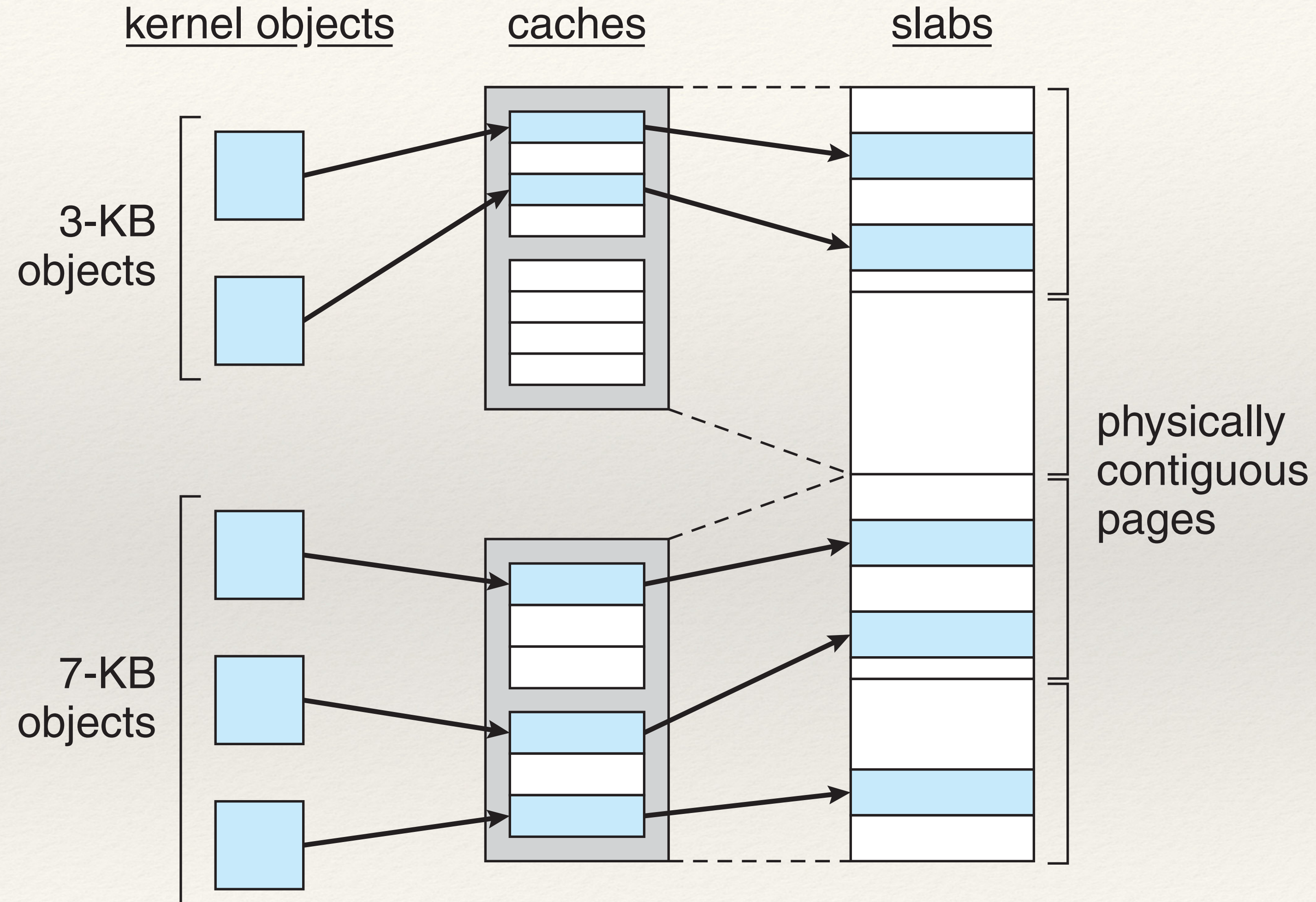
Buddy System

- ❖ Pro: Adjacent buddies can be combined to form larger contiguous segments using a technique known as **coalescing**
- ❖ Con: Internal Fragmentation

Slab Allocation

- ❖ Slab - made up of one or more physically contiguous pages
- ❖ Cache - consists of one or more slabs
- ❖ Objects - instantiations of the kernel data structure the cache represents

Slab Allocation



Slab Allocation - Example

- ❖ Process descriptor is of type `struct task_struct`
- ❖ When a new task is created, requests memory for a `task_struct` from the cache
- ❖ Cache will fulfill request using an object that has already been allocated in a slab and is marked as free.

Slab Allocation - Example

- ❖ Slab can be in three possible states:
 - ❖ Full - all objects in the slab are marked as used
 - ❖ Empty - all objects in the slab are marked as free
 - ❖ Partial - the slab consists of both used and free objects

Slab Allocation - Example

- ❖ Slab allocator first attempts to satisfy requests with
 1. A free object in a partial slab
 2. A free object in an empty slab
 3. A new slab is allocated and assigned to cache and memory for the object is allocated from the new slab.

Slab Allocation - Benefits

- ❖ No memory is wasted due to fragmentation
- ❖ Memory requests can be satisfied quickly

Virtual Memory

- ❖ Allocation of Frames
- ❖ Thrashing
- ❖ Memory-Mapped Files
- ❖ Allocating Kernel Memory
- ❖ Other Considerations

Other Considerations

- ❖ Prepaging - load pages into memory before they are needed
 - ❖ Useful for when a program is initially loaded so there isn't a high level of page-faults loading necessary pages into memory
- ❖ Page Size

Other Considerations

- ❖ Page Size
 - ❖ Memory is better utilized with small pages
 - ❖ Overhead is greater
 - ❖ I/O time is better with larger pages
 - ❖ Seek time, latency time, transfer time
 - ❖ Seek & Latency time are typically greater than the transfer time.

Other Considerations

- ❖ Page Size (Example from 9.1.1)
 - ❖ Latency is 8 ms
 - ❖ Seek time is 20ms
 - ❖ 0.2 ms to transfer 512 bytes

Other Considerations

- ❖ Page Size (Smaller)
 - ❖ Less internal fragmentation
 - ❖ Better resolution (load only what is needed)
 - ❖ Increased number of page faults tho...

Other Considerations

- ❖ Clearly
 - ❖ **Page Size (Smaller)**
 - ❖ Page Size (Larger)

Other Considerations

- ❖ Clearly
 - ❖ Page Size (Smaller)
 - ❖ **Page Size (Larger)**

TLB Hit Ratio

- ❖ TLB hit ratio - percentage of virtual address translations that are resolved in the TLB rather than the page table
- ❖ Increasing the entries in the TLB increase the hit ratio

TLB Reach



Operating System Essentials Silberschatz / Galvin / Gagne

Image retrieved from: <https://setupforvolleyball.com/beginner-drills-blocking/>

TLB Reach

- ❖ TLB - amount of memory accessible from the TLB
- ❖ Calculated by taking the number of entries multiplied by page size

$$TLBReach = tlbEntries \times pageSize$$

TLB Reach

- ❖ Working Set - pages that an application is frequently using
- ❖ Ideally the working set for an app is stored in the TLB

TLB Reach

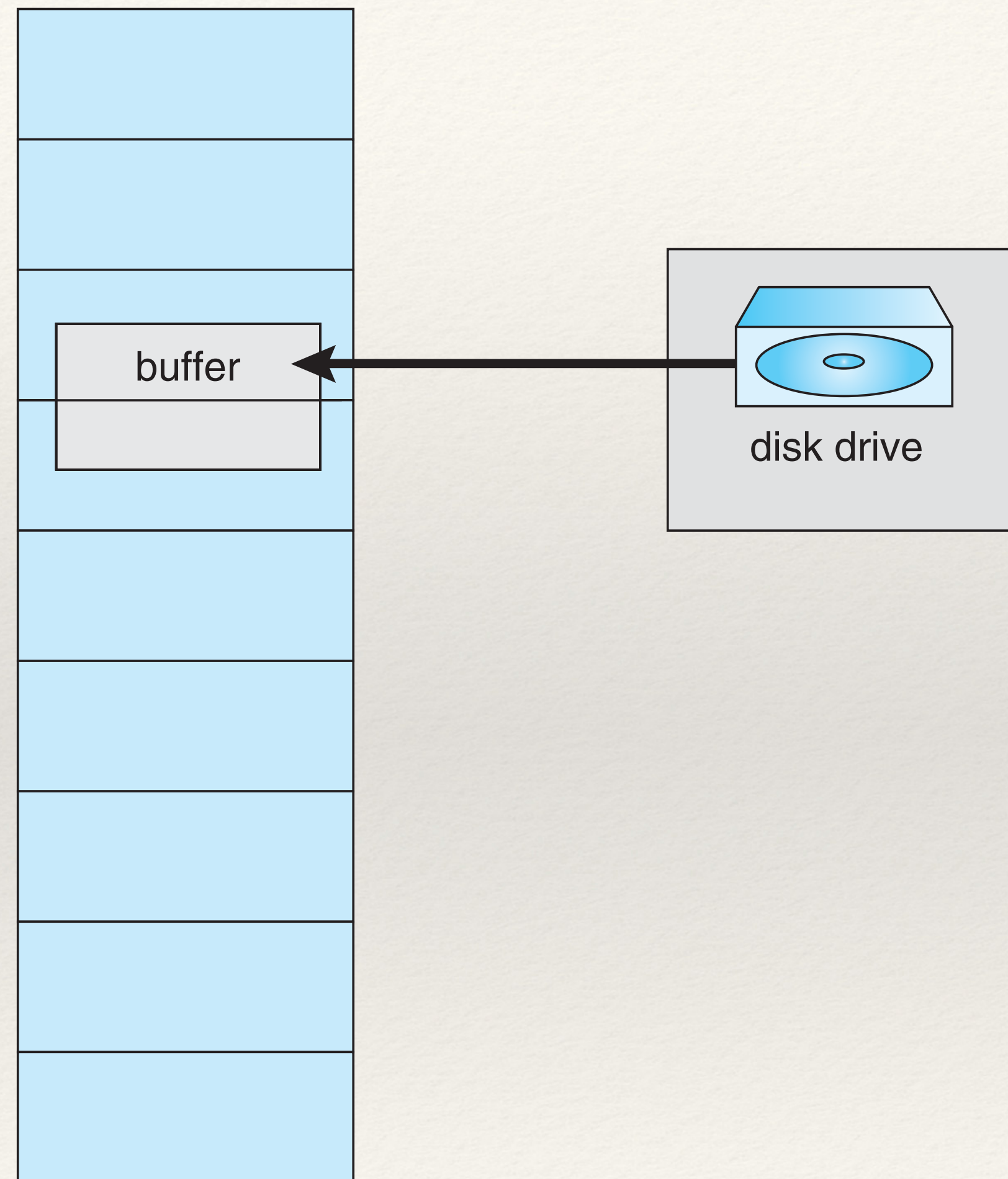
- ❖ How to increase the TLB reach?
 - ❖ Double the page size
 - ❖ Allow multiple page sizes

Program Structure

```
int i, j;  
int[128][128] data;  
  
for (j = 0; j < 128; j++)  
    for (i = 0; i < 128; i++)  
        data[i][j] = 0;
```

```
int i, j;  
int[128][128] data;  
  
for (i = 0; i < 128; i++)  
    for (j = 0; j < 128; j++)  
        data[i][j] = 0;
```


I/O Interlock and Page Locking



I/O Interlock and Page Locking

- ❖ Lock bits - used to lock a page into memory
 - ❖ Process could be waiting for I/O
 - ❖ Could be abused

END
